SDLC PROCESS MODEL ANALYSIS AND SELECTION

for

ROS2-Based Autonomous Warehouse Fleet Management System

Implementing a Dynamic Lane Reservation Strategy for Multi-Robot Traffic Coordination

Prepared by	Diya Jabin
Register Number	24BRS1172
Programme	B.Tech. Computer Science and Engineering
Specialization	Artificial Intelligence and Robotics
Course	Software Engineering Lab
Institution	VIT Chennai

1. Problem Analysis

The proposed software application is a ROS2-based Autonomous Warehouse Fleet Management System (AWFMS). It is intended to coordinate multiple autonomous mobile robots operating in a simulated warehouse environment. The system manages robot registration, delivery task allocation, real-time fleet monitoring, dynamic lane reservation, traffic conflict prevention, dashboard visualization, event logging, and performance analytics.

The project contains several independent but interconnected modules and relies on iterative experimentation with ROS2, Gazebo, navigation, communication, and multi-robot coordination. During development, requirements and design decisions may need refinement based on simulation results, integration issues, and evaluator feedback. Therefore, the selected development process should support incremental delivery, regular testing, technical risk reduction, and flexible change management.

2. Study of SDLC Process Models

2.1 Waterfall Model

A sequential model in which requirements, design, implementation, testing, deployment, and maintenance are completed in a fixed order. It is easy to understand and document, but changes introduced after development begins are expensive. It is less suitable for this project because robot coordination and simulation requirements may evolve during implementation.

2.2 Spiral Model

A risk-driven iterative model that combines prototyping and systematic development. Each cycle includes planning, risk analysis, engineering, and evaluation. It is useful for technically risky systems, but its formal risk-management activities may be too complex and time-consuming for a semester-scale academic project.

2.3 Agile Model

An iterative and incremental model in which the product is developed through short cycles called sprints. Each sprint delivers a tested increment and incorporates feedback. Agile is well suited to modular systems, evolving requirements, continuous integration, and frequent validation.

2.4 Prototype Model

A preliminary version is developed to clarify uncertain requirements and obtain feedback before building the final system. It is useful for validating the Streamlit dashboard and robot-traffic interactions, but by itself it does not provide a complete framework for managing the full development lifecycle.

2.5 Incremental Model

The system is divided into smaller functional increments that are implemented and delivered one after another. This is suitable for the modular nature of AWFMS. However, Agile extends the incremental approach by adding regular feedback, sprint planning, review, and adaptation.

2.6 V-Model

A verification-and-validation model in which each development phase has a corresponding testing phase. It provides strong traceability and disciplined testing, but it assumes relatively stable requirements and is less flexible when simulation findings require frequent changes.

3. Comparative Analysis

Model	Change Flexibility	Testing	Management Overhead	Suitability for AWFMS
Waterfall	Low	Late	Low	Low
Spiral	High	Continuous	High	Moderate
Agile	High	Continuous	Moderate	Very High
Prototype	High	Early	Moderate	Moderate
Incremental	Moderate	Per increment	Moderate	High
V-Model	Low	Mapped to phases	Low	Low

Based on this comparison, Agile provides the best balance of flexibility, continuous testing, manageable project overhead, and incremental delivery for the proposed system.

4. Selected Process Model: Agile

The Agile SDLC process model is selected for this project.

4.1 Justification

- **Modular development:** The system can be divided into robot registration, task management, monitoring, traffic coordination, dashboard, database, and analytics modules. These modules can be implemented as separate sprint increments.
- **Evolving technical requirements:** Multi-robot simulation may reveal communication, navigation, deadlock, lane-reservation, and performance issues that require changes to requirements or design.
- **Early working software:** A basic fleet manager and simulated robot can be demonstrated early, after which additional robots, traffic rules, and dashboard capabilities can be added.
- **Continuous testing and integration:** ROS2 nodes, services, actions, database components, and the dashboard must be tested repeatedly as they are integrated. Agile supports testing within every sprint.
- **Regular feedback:** Sprint reviews allow project guides, evaluators, and team members to provide feedback before the next increment is developed.
- **Reduced project risk:** High-risk features such as dynamic lane reservation and multi-robot conflict handling can be implemented and validated early rather than postponed until the end.
- **Appropriate for semester constraints:** Short, time-boxed sprints make it easier to prioritize essential features and maintain visible progress within the academic schedule.

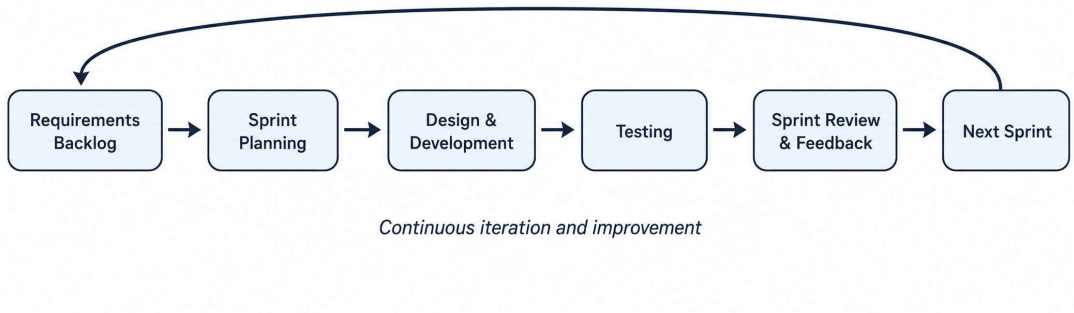

Figure 1. Agile development cycle proposed for the AWFMS project

5. Proposed Agile Sprint Plan

Sprint	Primary Objective	Expected Increment
Sprint 1	Requirement analysis, scope finalization, ROS2 workspace setup, warehouse layout, initial architecture.	Approved backlog and architecture
Sprint 2	Robot registration, unique robot identification, fleet initialization, status communication.	Registered and monitored robot fleet
Sprint 3	Delivery request creation, task validation, robot allocation, task-state management.	Working delivery-task flow
Sprint 4	Fleet monitoring, robot-location updates, event logging, operational status visualization.	Real-time fleet monitoring
Sprint 5	Dynamic lane reservation, waiting-state logic, lane release, conflict and deadlock handling.	Coordinated multi-robot lane usage
Sprint 6	Streamlit dashboard, SQLite integration, task history, alerts, and analytics.	Integrated operator dashboard
Sprint 7	System integration, performance and safety testing, defect correction, documentation, final demonstration.	Tested final system and documentation

6. Agile Practices to Be Followed

- Product backlog containing user stories, functional requirements, defects, and technical tasks.
- Sprint planning to select achievable work for each iteration.
- Incremental implementation using Git and GitHub for version control.
- Unit, integration, simulation, and acceptance testing during every sprint.
- Sprint review to demonstrate the working increment and collect feedback.
- Sprint retrospective to identify improvements for the following sprint.
- Requirements traceability between SRS requirements, implementation tasks, and test cases.

7. Conclusion

After studying Waterfall, Spiral, Agile, Prototype, Incremental, and V-Model, the Agile model is identified as the most suitable process model for the ROS2-Based Autonomous Warehouse Fleet Management System. Its iterative and incremental nature supports modular implementation, frequent testing, rapid feedback, and adaptation to issues discovered during multi-robot simulation. The project will therefore be developed through a sequence of short sprints, with a tested and demonstrable software increment produced at the end of each sprint.

8. References

1. IEEE Computer Society, IEEE Recommended Practice for Software Requirements Specifications, IEEE Std. 830-1998.

2. Open Robotics, ROS 2 Documentation.

3. Open Robotics, Gazebo Documentation.

4. K. Beck et al., Manifesto for Agile Software Development, 2001.